Quase scale-to-zero no OKE: manter um pod aquecido e deixá-lo mínimo

Relatório de pesquisa, 27 de outubro de 2026, revisão 2. Versão em inglês: report-en-light.pdf.

Escopo: como obter a maior parte da economia do scale-to-zero no Oracle Kubernetes Engine (OKE) sem pagar o custo do cold start (partida a frio). O relatório cobre o escalonamento vertical do pod que continua rodando, os recursos do OKE e do Kubernetes que já ajudam, se reduzir CPU ou memória economiza mais, e como a resposta muda para Node.js, PHP e Go.

Todos os manifestos citados estão em `examples/`. As afirmações marcadas com **(lab)** foram verificadas em um cluster local Kubernetes 1.36.4 com `lab/run-lab.sh` e `lab/run-runtime-lab.sh`. Os números de custo vêm de `lab/cost-model.py` e da tabela pública de preços da OCI.

Unidades: as quantidades de CPU e memória usam a notação do Kubernetes. `50m` significa 50 milicores, 5% de uma CPU; `512Mi` significa 512 mebibytes, meio gibibyte. O Apêndice C explica todas as unidades e mostra passo a passo como os custos mensais são calculados.

Termos mantidos em inglês por serem os nomes usados no Kubernetes: *requests* (recursos reservados), *limits* (teto), *node* (nó de trabalho), *eviction* (despejo do pod pelo kubelet), *OOM kill* (processo morto por falta de memória).

Mudanças na revisão 2, depois das respostas do time:

- Orientações por runtime para Node.js, PHP e Go, com medições do lab (seção 7).
- Densidade de pods por nó, que limita o empacotamento quando os pods ficam pequenos (seção 9.5).
- Como lidar com tráfego imprevisível, com uma consulta que verifica a hipótese do horário comercial (seção 10).
- O que pode ir a zero, e o que continua custando dinheiro em zero (seção 11).
- Comparação das variantes de cluster do OKE: basic ou enhanced, tipos de nó, autoscalers de nó, rede de pods, versão do Kubernetes (seção 12).
- Novos exemplos 11a a 15. Correção no exemplo 05: ele falhava em pods com mais de um container.

Acrescentado em 28 de outubro: Apêndice C, um glossário das unidades usadas para CPU, memória e preços, com um exemplo de custo calculado passo a passo.

Sumário

1. Resumo ... 4
2. Contexto e premissas ... 5
3. Como o dinheiro flui no OKE ... 5
4. O que o OKE e o Kubernetes já oferecem ... 6
5. Escalonamento vertical no OKE ... 7
5.1 In-place pod resize: o mecanismo ... 7
5.2 Vertical Pod Autoscaler ... 8
5.3 CPU startup boost: regime pequeno, inicialização rápida ... 9
5.4 Encolher o pod aquecido nos períodos ociosos ... 9
5.5 Regras de interação ... 10
6. CPU ou memória: qual redução economiza mais? ... 10
6.1 Preço por unidade ... 10
6.2 Para onde vai o dinheiro em um pod (E5 Flex, 24x7) ... 11
6.3 O que acontece quando cada um fica baixo demais ... 11
6.4 O comportamento em repouso é diferente ... 11
6.5 Comparação de cenários (30 serviços, E5 Flex) ... 12
6.6 Conclusão ... 12
7. Orientações por runtime: Node.js, PHP e Go ... 12
7.1 Node.js ... 13
7.2 PHP ... 13
7.3 Go ... 14
7.4 Cold start: o que os números do lab significam ... 15
8. Camada horizontal: nunca zero, raramente mais que o necessário ... 15
9. Camada de nós: transformar requests menores em uma conta menor ... 16
9.1 Cluster Autoscaler (add-on do OKE) ... 16
9.2 Karpenter Provider for OCI ... 16
9.3 Consolidação significa cold starts ... 17
9.4 Alavancas de preço de nó para o pod aquecido (50m / 1Gi, 24x7) ... 17
9.5 Densidade de pods: quando endereços IP, e não CPU ou memória, limitam o empacotamento ... 17
10. Tráfego imprevisível ... 18
10.1 Deixar um palpite errado barato ... 18
10.2 Capacidade de reserva com pods placeholder ... 19
10.3 Verificar a hipótese do horário comercial ... 19
10.4 O que fazer com o resultado ... 19
11. O que pode ir a zero ... 20
11.1 O que "zero" significa ... 20
11.2 Candidatos ... 20
11.3 O que continua custando dinheiro em zero ... 21
11.4 Obter os números por ambiente ... 21
12. Escolha da variante de cluster do OKE ... 21
12.1 Cluster basic ou enhanced ... 21
12.2 Tipos de nó ... 22
12.3 Autoscaler de nós ... 22
12.4 Rede de pods (CNI) ... 23
12.5 Versão do Kubernetes ... 23
12.6 Se vocês puderem escolher ... 23
13. Plano de adoção sugerido ... 24
14. Riscos e ressalvas ... 24
15. Perguntas em aberto para o time ... 25
Apêndice A. Arquivos ... 25
Apêndice B. Fontes ... 26
Apêndice C. Glossário de unidades ... 28
C.1 CPU ... 28
C.2 Memória ... 28
C.3 Preços e tempo ... 29

C.4 Exemplo calculado: quanto custa um pod aquecido 29
C.5 Outros termos usados com números 29

1. Resumo

1. **A economia vem dos requests, não do uso.** Nos managed nodes do OKE você paga por nós inteiros. O scheduler posiciona os pods pelos *requests*, e o autoscaler de nós só remove um nó quando os requests permitem. Encolher um pod só economiza se requests menores resultarem em menos nós ou em nós mais baratos.
2. **Um pod aquecido com CPU reduzida captura cerca de 90% da economia do scale-to-zero.** No modelo de custo (30 serviços, 3 réplicas no horário comercial), o scale-to-zero de verdade à noite economiza 64%. Manter 1 réplica por serviço com o request de CPU reduzido para 50m economiza 58%. O pod aquecido custa cerca de US$ 1,40 por serviço por mês (E5 Flex, pay-as-you-go).
3. **Escalonamento vertical sem reinício já está pronto no OKE.** O in-place pod resize é GA desde o Kubernetes 1.35. O OKE suporta hoje 1.34.10, 1.35.2 e 1.36.4 (1.37.0 em preview). O VPA 1.6+ aplica recomendações sem recriar o pod (`InPlaceOrRecreate`), e o VPA 1.7+ traz um CPU Startup Boost em alpha.
4. **Corte CPU primeiro, memória com cuidado.**
   - No E5/E6 Flex, 1 vCPU custa o mesmo que 7,5 GB de RAM, então a CPU representa cerca de 78% do custo de um pod típico de 500m/1Gi.
   - CPU é compressível: um container sem CPU suficiente fica lento, ninguém o mata. Em repouso, o uso de CPU cai de fato para poucos millicores.
   - Memória não é compressível. Passar do *limit* de memória causa OOM kill do container. Usar mais que o *request* de memória coloca o pod no início da fila de eviction quando falta memória no nó.
   - Se a memória cai em repouso depende do runtime. No lab, o PHP-FPM com `pm = ondemand` foi de 172 MiB para 10 MiB em 25 s depois da última requisição. Node.js e Go devolveram a memória do pico depois de cerca de 2 e 4,5 minutos. Caches de longa duração nunca caem.
   - Regra: reduza os requests de CPU agressivamente. Reduza os *requests* de memória só até o uso medido em repouso. Nunca reduza o *limit* de memória in-place.
5. **Node.js, PHP e Go leem os limits do container de formas diferentes** (seção 7):
   - O Node.js define o limite do heap a partir do limit de memória uma única vez, na inicialização (512Mi dá um heap de 259 MiB). Um resize in-place não o altera, então mude limits de memória só com um rollout.
   - O PHP-FPM com `pm = ondemand` fica em repouso só com o processo master e o OPcache, cerca de 10 MiB no lab.
   - O Go 1.25+ define o GOMAXPROCS pelo limit de CPU e acompanha mudanças in-place em cerca de 1 s, se o `go.mod` declarar `go 1.25` ou superior. O GOMEMLIMIT precisa ser definido manualmente.
   - Acordar do zero um servidor mínimo de qualquer um dos três, em um nó que já tem a imagem, levou de 0,8 a 2,1 s. Aplicações reais somam o próprio tempo de inicialização, e um nó novo soma minutos.
6. **Tráfego imprevisível é administrável se você encolhe requests, não limits** (seção 10). Um pod com request de 25m e limit de 1 CPU ainda usou de 0,91 a 0,99 CPU em um nó com CPU livre. Escale horizontalmente com métricas absolutas, mantenha espaço livre nos nós com pods placeholder, e confira a hipótese do horário comercial com a consulta da seção 10.3 antes de depender de agendas.
7. **"Zero" significa primeiro zero pods, depois zero nós** (seção 11). Bons candidatos: ambientes de não-produção fora do expediente, workers de fila em todos os ambientes e ferramentas internas pouco usadas. Em zero você ainda paga o control plane de um cluster enhanced (US$ 73/mês), o node pool de sistema (cerca de US$ 71/mês para dois nós pequenos), load balancers além do gratuito e volumes.
8. **Stack recomendada:**
   - KEDA (ou HPA) com `minReplicas: 1` e alvo de CPU absoluta ou de taxa de requisições para serviços com usuários.
   - VPA em modo `Off` para medir, depois `InPlaceOrRecreate` com `RequestsOnly`, ou um encolhedor in-place por agenda para fora do expediente.
   - Um CPU startup boost onde a inicialização depende de CPU.

    - GoKubeDownscaler para namespaces inteiros de não-produção, e KEDA com `minReplicaCount: 0` para workers de fila.
    - Cluster Autoscaler (add-on do OKE) ou Karpenter Provider for OCI para transformar os requests liberados em menos nós. Nós burstable para pools de pouca CPU. Nós preemptible só para réplicas extras.
9. **Variante de cluster, se vocês puderem escolher** (seção 12): cluster enhanced, Kubernetes 1.36, managed nodes e o CNI flannel, a menos que precisem de IPs de pod dentro da VCN. Com a rede VCN-native, um nó de 2 OCPUs comporta no máximo 31 pods, o que para pods pequenos custa mais que a CPU e a memória deles. O CNI não pode ser trocado depois que o cluster é criado.
10. **Evite:**
    - Virtual nodes para pods pequenos aquecidos. A alocação arredonda para OCPUs e GBs inteiros, então um pod de 50m custa cerca de 10x mais lá.
    - HPA por *Utilization* de CPU junto com redução de requests. Utilization é uso dividido pelo request, então encolher o request dispara uma avalanche de scale-out.
    - Dois controladores alterando os recursos do mesmo pod.
    - Colocar a única réplica aquecida em capacidade preemptible.

2. Contexto e premissas

- **Contexto do time (respostas de 27/10/2026):**
    - Runtimes: principalmente Node.js, além de PHP e Go. A plataforma deve suportar o que os usuários precisarem.
    - Tráfego: imprevisível por enquanto. A expectativa é de picos no horário comercial, mas isso não está confirmado.
    - Ainda não existe divisão de custo por ambiente.
    - Tipo de cluster, versão do Kubernetes e autoscaler de nós ainda não foram definidos.
- **Versões do OKE (out/2026):** 1.34.10, 1.35.2, 1.36.4, além de 1.37.0 em preview (release notes do OKE). O in-place resize exige 1.33+ e é GA a partir da 1.35. O Kubernetes 1.35+ exige cgroup v2, fornecido pelas imagens OKE OL8 (blog OKE 1.35).
- **Tipo de cluster:** assumo clusters *enhanced* (US$ 0,10/h por cluster). Os add-ons gerenciados (Cluster Autoscaler, Metrics Server), virtual nodes, workload identity e self-managed nodes exigem esse tipo. A seção 12 compara clusters basic e enhanced.
- **Horário comercial:** segunda a sexta, 08:00 às 20:00 America/Sao_Paulo (UTC−3, sem horário de verão), 60 das 168 horas da semana.
- **Preços:** API pública de preços da OCI, pay-as-you-go em USD, consultada em 27/10/2026. Descontos negociados mudam os valores absolutos, não as proporções.
- **Simplificação do modelo de custo:** o custo é atribuído aos recursos reservados (requests). Isso supõe nós bem empacotados e um autoscaler de nós que remove capacidade. A economia real é menor na proporção da sua ineficiência de empacotamento.
- **Lab:** kind v0.33 com Kubernetes 1.36.4 sobre containerd 2.3.4, VPA 1.8.1, KEDA 2.21.0 com o HTTP add-on 0.16.0, GoKubeDownscaler 1.4.2, metrics-server 0.9.0. Runtimes: Node.js 24.22.1 e 26.12.1, PHP 8.4 FPM, Go 1.27. O OKE usa CRI-O em vez de containerd, e o lab não testa o provisionamento de nós da OCI, então só o comportamento no nível do pod foi verificado.

3. Como o dinheiro flui no OKE

Três camadas definem a conta. "Quase zero" significa trabalhar as três.

Camada	O que muda	Ferramentas	Economiza quando
Réplicas (horizontal)	Número de pods	HPA, KEDA	Menos pods significa menos requests
Tamanho do pod (vertical)	Requests/limits por pod	In-place resize, VPA, startup boost, encolhedor	Requests menores
Capacidade (nós)	Número, shape e preço dos nós	Cluster Autoscaler, Karpenter (KPO), shapes flex/burstable/preemptible/Arm	Nós são removidos ou trocados por mais baratos

As duas primeiras camadas só reduzem *requests*. É na terceira que requests viram dinheiro. Se nenhum nó sai, encolher pods não economiza nada.

4. O que o OKE e o Kubernetes já oferecem

Recurso	Situação no OKE (out/2026)	Papel no "quase zero"	Observações
In-place pod resize	GA no OKE 1.35.2 / 1.36.4; beta e ligado por padrão no 1.34.10	Encolher e aumentar pods em execução sem reinício	Só CPU e memória; a classe de QoS não pode mudar
Vertical Pod Autoscaler	Você instala (Helm chart ou script upstream). Guia do OKE	Medir e ajustar requests	Não funciona em virtual nodes. O guia do OKE ainda usa o VPA 0.8 e o modo `Auto`, que está deprecated; use o VPA 1.8.x
HPA	Nativo	Manter `minReplicas: 1`, escalar com a carga	Use `AverageValue`, não `Utilization`, quando os requests mudam
HPA scale to zero	Beta no K8s 1.37 (blog)	Zero de verdade em não-produção	O OKE tem a 1.37 só em preview
Metrics Server	Add-on do OKE	Necessário para HPA e VPA	
Cluster Autoscaler	Add-on do OKE ou standalone (docs)	Remove nós quando os requests caem; pools podem ir a 0	Um shape por node pool; mantenha um pool de sistema fora do controle dele
Karpenter Provider for OCI (KPO)	GA em abril/2026, v1.4.0 (docs, blog do GA)	Consolida nós subutilizados, escolhe shapes, suporta burstable e preemptible	Suporta K8s 1.31 a 1.36
Shapes flexíveis	Managed nodes e KPO	Escolher a proporção OCPU:memória que casa com seus requests	1 a 64 GB por OCPU no E4/E5
Instâncias burstable	KPO (`baselineOcpuUtilization`) e self-managed nodes	Pagar 12,5% ou 50% das OCPUs	Memória é cobrada inteira; o burst dura no máximo cerca de 1 h (docs)
Capacidade preemptible	Managed node pools e KPO	Nós 50% mais baratos	Pode ser retomada a qualquer momento, com aviso de cerca de 2 min; não pode ser burstable (docs)
Ampere A1/A2 (Arm)	Managed nodes e KPO	CPU mais barata	Exige imagens multi-arch
Virtual nodes	Clusters enhanced	Capacidade serverless para picos	Não serve para pods pequenos (seção 9.4)

Não encontrei campo para o baseline burstable na API de managed node pools do OKE: o `NodeShapeConfig` só expõe `ocpus` e `memoryInGBs`. Por isso, workers burstable vêm pelo KPO ou por self-managed nodes.

Ferramentas open source relacionadas:

Ferramenta	Versão verificada	Uso
KEDA	2.21.0	Gatilhos por cron e eventos; permite `minReplicaCount: 1`. A Oracle mostra KEDA com Cluster Autoscaler no OKE neste blog
KEDA HTTP add-on	0.16.0 (beta)	Segura as requisições enquanto o serviço sobe do zero. Só serve para não-produção, porque ainda espera o cold start
kube-startup-cpu-boost	0.21.3	Boost temporário de CPU na inicialização via in-place resize
Goldilocks	4.16.2	Dashboard sobre as recomendações do VPA
KRR	1.31.1	Recomendações de requests a partir do histórico do Prometheus, sem VPA
OpenCost	1.121.3	Custo por namespace e por workload; suporta preços da OCI
GoKubeDownscaler / kube-green	1.4.2 / 0.7.1	Scale-to-zero fora do expediente em não-produção
Elastic CPU Pod Autoscaler (DoorDash)	n/d	Autoscaling in-place rápido, só de CPU

Produtos comerciais também implementam diretamente a ideia de "encolher a última réplica". Um exemplo é o PodResourceProfile da Kedify, que redimensiona os requests in-place quando um ScaledObject do KEDA fica inativo. CAST AI, ScaleOps e outros também oferecem rightsizing in-place. Não os avaliei.

5. Escalonamento vertical no OKE

5.1 In-place pod resize: o mecanismo

Você altera `spec.containers[*].resources` pelo subresource `resize` do pod, e o kubelet aplica os novos valores ao cgroup do container em execução (docs do Kubernetes).

```
kubectl -n demo patch pod my-app-xxxx --subresource resize \
  -p '{"spec":{"containers":[{"name":"app","resources":{"requests":{"cpu":"25m","memory":"64Mi"}}}]}}'
```

Valor	Efeito no cgroup v2 do Linux	Efeito de reduzir
Request de CPU	`cpu.weight` (fatia sob disputa)	Menos CPU só quando o nó está ocupado; o scheduler vê mais espaço livre
Limit de CPU	`cpu.max` (cota rígida)	Throttling acima do novo limit
Request de memória	Só agendamento e ordem de eviction (a menos que o feature gate `MemoryQoS` esteja ligado)	O scheduler vê mais espaço livre; o pod sobe na fila de eviction se uso > request
Limit de memória	`memory.max`	OOM kill se o uso passar dele

Regras que importam para esse padrão:

- **A classe de QoS não pode mudar.** Um pod Guaranteed (requests = limits) só pode ser redimensionado mantendo requests iguais aos limits. Para baixar requests mantendo os limits, o pod precisa nascer Burstable (`examples/01-app-deployment.yaml`).
- **`resizePolicy`** por recurso é `NotRequired` (padrão, sem reinício) ou `RestartContainer`.
- **Reduzir o limit de memória é best effort.** Se o uso atual estiver acima do novo limit, o kubelet mantém o resize "em andamento" e não o aplica.
- **Use strategic merge patch (padrão do kubectl) ou JSON patch.** `--type merge` substitui a lista inteira de containers, e o API server rejeita o pedido. **(lab)**
- **O template do Deployment não muda.** Pods novos (scale-out, reagendamento após drain de nó) nascem com o tamanho do template. Ferramentas de GitOps não veem drift.

- O RBAC da automação precisa de `get` e `patch` em `pods/resize`. **(lab)**
- O Kubernetes 1.36 traz em beta o in-place resize de recursos no nível do pod (`spec.resources`), útil para pods com vários containers (blog).

Resultados do lab (Kubernetes 1.36.4):

Teste	Resultado
Requests 500m/256Mi → 25m/64Mi, limits inalterados	Aplicado, `restartCount` 0. `cpu.weight` foi de 59 para 6; `cpu.max` e `memory.max` inalterados
Requests iguais aos limits (Burstable → Guaranteed)	Rejeitado: "Pod QOS Class may not change as a result of resizing"
Request de memória 64Mi em container usando ~200Mi	Aplicado, sem reinício
Limit de memória 400Mi → 128Mi em container usando ~200Mi	Não aplicado; condição `PodResizeInProgress`; o limit real continuou 400Mi
Limit de memória 400Mi → 300Mi (acima do uso)	Aplicado, sem reinício
Requests de volta para 500m/256Mi	Aplicado, sem reinício

Outra observação do lab: depois de reduzir o request de memória, o `oom_score_adj` do container continuou com o valor calculado a partir do request original. Não conte com um request menor tornando o pod uma vítima mais provável do OOM killer do kernel. A ordem de eviction do kubelet, por outro lado, usa o request atual.

5.2 Vertical Pod Autoscaler

Instalação no OKE com o Helm chart oficial, em vez do script da época do VPA 0.8 que aparece no guia do OKE:

```
helm repo add autoscaler https://kubernetes.github.io/autoscaler
helm upgrade --install vpa autoscaler/vertical-pod-autoscaler --version 0.15.0 -n kube-system
# Métricas: habilite antes o add-on "Kubernetes Metrics Server" do OKE.
```

Modos de atualização no VPA 1.8 (referência da API):

Modo	Comportamento	Quando usar
`Off`	Só recomenda	Sempre comece por aqui (`02`)
`Initial`	Define requests só na criação do pod	Em conjunto com outro controlador que redimensiona depois
`Recreate`	Faz eviction dos pods para aplicar mudanças	Legado; `Auto` está deprecated e significa o mesmo
`InPlaceOrRecreate`	Redimensiona in-place e só faz eviction se falhar	Escolha padrão (GA desde o VPA 1.6) (`03`)
`InPlace`	Redimensiona in-place, nunca faz eviction	Alpha (VPA 1.7, `--feature-gates=InPlace=true`)

Política recomendada para este caso (de `examples/03-vpa-inplace.yaml`):

```
spec:
  updatePolicy:
    updateMode: "InPlaceOrRecreate"
    minReplicas: 1                    # permite atualizar workloads com uma réplica
  resourcePolicy:
    containerPolicies:
      - containerName: app
        controlledValues: RequestsOnly # mantém os limits como escritos
        minAllowed: { cpu: 25m, memory: 128Mi }
        maxAllowed: { cpu: "2", memory: 2Gi }
```

`RequestsOnly` mantém o limit de memória como teto fixo de segurança. Também mantém o limit de CPU, que a JVM e outros runtimes leem na inicialização para dimensionar seus pools de threads.

No que o VPA é bom e no que não é. O recommender mantém um histograma de uso de 8 dias com decaimento exponencial (flags):

- O alvo de CPU é o percentil 90 do uso.
- O alvo de memória é o percentil 90 dos *picos* diários.
- Margem de segurança de 15% sobre os dois.
- Pisos de 25m de CPU e 250 MiB de memória por pod.

Ou seja, o VPA dimensiona o pod para o dia de pico. Ele não encolhe o pod à noite. Use o VPA como ferramenta de rightsizing e um mecanismo separado (5.4) para os períodos ociosos.

Resultados do lab:

- Com o workload "hamster" do upstream (request de 100m, consome cerca de 500m), o `InPlaceOrRecreate` subiu o request para 587m em cerca de dois minutos, no mesmo pod e sem reinícios.
- O VPA também subiu o request de memória de 50Mi para 250Mi por causa do piso padrão de 250 MiB (a flag se chama `-mb`, mas o lab mostrou o valor aplicado como `250Mi`). Para pods muito pequenos, reduza `--pod-recommendation-min-memory-mb` no recommender.

5.3 CPU startup boost: regime pequeno, inicialização rápida

Um request de CPU pequeno atrasa a inicialização quando o nó está ocupado, e um limit de CPU baixo atrasa sempre. Runtimes com JIT sofrem mais. O startup boost dá mais CPU ao pod até ele ficar Ready e depois o encolhe in-place. Assim um request mínimo em regime convive com um cold start curto, e o scale-out a partir de 1 réplica também fica mais rápido.

Opção	Situação	Exemplo
`startupBoost` do VPA	Alpha, VPA 1.7+, feature gate `CPUStartupBoost` no admission controller e no updater	`04-vpa-startup-boost.yaml`
Operador kube-startup-cpu-boost	v0.21.3, independente do VPA	`10-kube-startup-cpu-boost.yaml`

Resultados do lab:

- Boost do VPA com fator 4: o request de CPU foi de 500m para 999m na criação. O `RequestsOnly` limita o boost a logo abaixo do limit de 1 CPU para manter a classe de QoS. 30 s depois do Ready, o VPA redimensionou o pod in-place para a recomendação (25m), sem reinícios.
- O kube-startup-cpu-boost aplicou o boost na admissão e o reverteu 3 s depois, quando o pod ficou Ready.

Cuidados (README do kube-startup-cpu-boost):

- Se o boost aumentar o *limit* de CPU, a JVM dimensiona seus pools para a CPU aumentada e os mantém depois da reversão. Use boosts modestos ou defina `-XX:ActiveProcessorCount`.
- Um request aumentado pode disparar um scale-up de nó que fica inútil um minuto depois. Mantenha alguma folga ou deixe o scale-down mais lento.

5.4 Encolher o pod aquecido nos períodos ociosos

Duas formas:

a) Por agenda (horário comercial previsível). `examples/05-idle-shrinker-cronjob.yaml` é uma implementação pequena e sem dependências:

- Às 20:00, de segunda a sexta, um CronJob reduz os requests dos pods que optaram pelo recurso para os valores da anotação `idle-shrink.example.com/requests`.
- Às 07:30, um segundo CronJob restaura os requests a partir do template do ReplicaSet dono do pod.

- Os limits nunca são alterados.
- Proteção de memória: o job nunca define um request de memória abaixo do uso atual + 15%. Se não houver métricas, ele não mexe na memória.

Lab: encolher e restaurar funcionaram sem reinícios. A proteção de memória subiu um alvo ocioso de 32Mi para 230Mi em um container usando 200Mi.

b) Por evento (tráfego imprevisível). Encolher quando o ScaledObject fica inativo e aumentar na ativação. Ainda não existe um controlador open source maduro para isso. O Kedify PodResourceProfile faz isso comercialmente, e o EPCAS da DoorDash cobre o caso só de CPU. Outra opção: manter o pod aquecido num tamanho "pequeno mas responsivo" (por exemplo, 100m de CPU) e deixar o HPA adicionar pods de tamanho cheio. A seção 10 mostra como combinar agendas com tráfego imprevisível.

Não combine (a) com o VPA em modo `InPlaceOrRecreate`, `Recreate` ou `InPlace` no mesmo workload, porque o VPA desfaz a mudança. VPA em `Off` ou `Initial` não tem problema.

5.5 Regras de interação

Combinação	Seguro?	Por quê
HPA/KEDA por `Utilization` de CPU + qualquer redução de requests	Não	Exemplo: request de 500m com alvo de 70% escala em 350m. Depois de encolher para 25m, escala em 17,5m
HPA/KEDA por `AverageValue` de CPU, taxa de requisições ou tamanho de fila + VPA/encolhedor	Sim	O sinal não depende dos requests (`06`, `07`)
VPA (atualizando) + HPA na mesma métrica de CPU/memória	Não	Tanto o VPA upstream quanto o OKE desaconselham
VPA (atualizando) + encolhedor ocioso nos mesmos pods	Não	Dois escritores brigando
startupBoost do VPA + kube-startup-cpu-boost nos mesmos pods	Não	Escolha um
VPA (atualizando) + cron do KEDA com `minReplicaCount: 1`	Sim	Camadas diferentes

6. CPU ou memória: qual redução economiza mais?

6.1 Preço por unidade

No Kubernetes, `1` CPU = 1 vCPU. Nos shapes x86, 1 OCPU = 2 vCPUs. No Ampere A1, 1 OCPU = 1 vCPU. No A2, 1 OCPU = 2 cores. O Apêndice C.4 mostra passo a passo como esses preços viram os valores mensais abaixo.

Shape	USD por vCPU-hora	USD por GB-hora	GB de RAM que custam o mesmo que 1 vCPU
E4 Flex (AMD)	0,0125	0,0015	8,3
E5 Flex (AMD)	0,0150	0,0020	7,5
E6 Flex (AMD)	0,0150	0,0020	7,5
A1 Flex (Ampere)	0,0100	0,0015	6,7
A2 Flex (AmpereOne)	0,0070	0,0020	3,5

Serviços típicos pedem de 1 a 4 GB por vCPU, bem abaixo desses pontos de equilíbrio. A CPU é a maior parte da conta, a menos que o pod peça mais de cerca de 7 GB por vCPU nos shapes AMD.

6.2 Para onde vai o dinheiro em um pod (E5 Flex, 24x7)

Requests	CPU USD/mês	Memória USD/mês	Participação da CPU
1000m / 2Gi	10,95	3,14	78%
500m / 1Gi	5,47	1,57	78%
250m / 1Gi	2,74	1,57	64%
50m / 1Gi	0,55	1,57	26%

Depois que a CPU é reduzida, a memória vira a maior parte do que sobra. E a memória só pode ser reduzida além disso mudando a aplicação (6.4).

6.3 O que acontece quando cada um fica baixo demais

CPU baixa demais:

- Abaixo do request, nada acontece. Sob disputa, o container recebe CPU proporcional ao request, e a CPU livre é compartilhada.
- No limit, o kernel faz throttling do container: a latência sobe e nada é morto.
- Risco indireto: uma liveness probe que estoura o timeout por causa do throttling reinicia o container. Use timeouts generosos e não faça trabalho pesado dentro das probes.

Memória baixa demais. Três mecanismos diferentes podem matar ou mover um pod (node-pressure eviction):

Mecanismo	Gatilho	Resultado	Controlado por
OOM kill do container	Uso do container > **limit** de memória	O kernel mata o processo; o kubelet reinicia o container (cold start)	Limit de memória
Eviction pelo kubelet	Memória disponível do nó abaixo do limiar de eviction	O pod é apagado e recriado em outro lugar (cold start). Pods com uso acima do **request** saem primeiro, depois por prioridade, depois por quanto passam do request	Request de memória, PriorityClass
OOM do nó	O nó esgota a memória antes de o kubelet reagir	O kernel mata pelo `oom_score_adj`: BestEffort 1000, Burstable 2–999 (request menor = score maior), Guaranteed −997	Classe de QoS, request

Então a preocupação de que pods morrem com pouca memória está correta, e ela se divide em duas:

- **Limits decidem OOM kills.** Mantenha o limit de memória no pico + folga e nunca o reduza in-place abaixo do uso atual. O kubelet não aplica essa mudança **(lab)**, mas um pico de uso logo depois da verificação ainda é possível.
- **Requests decidem a ordem de eviction e o quanto o scheduler aperta os pods.** Um request de memória abaixo do uso real não libera RAM nenhuma. Ele só deixa o scheduler colocar mais pods no nó e transforma esse pod em uma das primeiras vítimas de eviction. Essa economia é falsa, a menos que o uso tenha caído de fato.

6.4 O comportamento em repouso é diferente

- **CPU:** um serviço ocioso usa poucos millicores. Baixar o request de 500m para 25–50m reflete a realidade, então a economia é real e de baixo risco.
- **Memória:** um serviço ocioso mantém o working set de longa duração (código carregado, caches, pools de conexão). A memória usada no último pico pode ou não voltar ao sistema operacional, dependendo do runtime. Medições do lab depois de um pico, sem tráfego adicional (seção 7):

Runtime	Logo depois do pico	Depois, sem tráfego
PHP-FPM, `pm = ondemand`	172 MiB	10 MiB depois de 25 s
PHP-FPM, `pm = static`	172 MiB	172 MiB, não cai

Runtime	Logo depois do pico	Depois, sem tráfego
Node.js 24	cerca de 200 MiB	cerca de 60 MiB depois de uns 2 min
Go 1.27	157 MiB	10 MiB depois de uns 4,5 min

- O Kubernetes não consegue fazer o processo devolver memória; quem decide são as configurações do runtime. Então meça a memória em repouso alguns minutos depois que o tráfego para, e defina o request de memória fora do expediente a partir desse número.
- Outros runtimes que a plataforma pode receber: a JVM dimensiona o heap por `-Xmx` ou `MaxRAMPercentage` na inicialização, e o G1 devolve heap não usado em repouso com `-XX:G1PeriodicGCInterval` (ZGC e Shenandoah fazem isso por padrão). O Server GC do .NET mantém heaps grandes; DATAS (padrão desde o .NET 9) ou `GCConserveMemory` os reduzem.
- Por isso, o in-place resize de memória serve principalmente para os *requests* (agendamento), não para o consumo real do processo.

6.5 Comparação de cenários (30 serviços, E5 Flex)

Horário comercial: segunda a sexta, 08:00–20:00 (60 h/semana), com 3 réplicas × (500m, 1Gi). O resto da semana (108 h) varia:

Estratégia fora do expediente	USD/mês	Economia vs sempre ligado
A. Sem mudança, 3 réplicas	634	0%
B. 1 réplica, tamanho cheio	362	43%
C. 1 réplica, request de CPU 50m, memória 1Gi	267	**58%**
D. 1 réplica, CPU 500m, request de memória 512Mi	347	45%
E. 1 réplica, CPU 50m + memória 768Mi	260	59%
F. 0 réplicas (scale-to-zero de verdade)	226	64%

Cortar a memória pela metade (D) acrescenta 2 pontos sobre B. Cortar a CPU (C) acrescenta 15. C chega a cerca de 90% da economia de F sem nenhum cold start.

6.6 Conclusão

1. **A CPU economiza mais e é mais segura de cortar.** Reduza os requests de CPU em repouso para 25–50m. Mantenha um limit de CPU nos serviços JVM (ele fixa a contagem de processadores que a JVM enxerga), ou defina `-XX:ActiveProcessorCount`. Para outros runtimes, o limit é opcional.
2. **Memória:** request = working set medido em repouso + 10–15%; limit = pico + folga. Reduções adicionais vêm de ajustes no runtime, não do Kubernetes.
3. **Depois de cortar a CPU, a memória domina o custo restante.** Escolha shapes com mais GB por OCPU (shapes flex permitem até 64 GB/OCPU) para não pagar CPU ociosa nos próprios nós.

7. Orientações por runtime: Node.js, PHP e Go

A plataforma vai receber principalmente Node.js, além de PHP e Go. Cada runtime lê os limits do container do seu jeito, e isso define o que pode ser alterado in-place com segurança. Nenhum dos três olha os *requests* de CPU ou memória, e é por isso que encolher requests é seguro para todos eles.

	Node.js 24	PHP-FPM 8.4	Go 1.27
Lê o limit de CPU	Sim: `os.availableParallelism()`, acompanha mudanças in-place **(lab)**	Não; a concorrência é `pm.max_children`	Sim: GOMAXPROCS = limit de CPU (mínimo 2), acompanha mudanças in-place em cerca de 1 s **(lab)**

	Node.js 24	PHP-FPM 8.4	Go 1.27
Lê o limit de memória	Sim, uma vez: o heap do V8 fica em cerca de 50% do limit na inicialização; não é recalculado depois de um resize **(lab)**	Não	Não; defina o `GOMEMLIMIT` você mesmo
Memória em repouso depois de um pico **(lab)**	cerca de 200 → 60 MiB depois de uns 2 min	`ondemand`: 172 → 10 MiB em 25 s; `static`: fica em 172 MiB	157 → 10 MiB depois de uns 4,5 min
Acordar do zero, servidor mínimo, imagem já no nó **(lab)**	0,8–1,9 s	1,1–1,4 s (com nginx)	1,0–2,1 s
Seguro mudar in-place	Requests e limits de CPU, requests de memória	Requests e limits de CPU, requests de memória	Requests e limits de CPU, requests de memória
Mudar só com rollout	Limit de memória (o tamanho do heap depende dele)	Limit de memória, junto com `pm.max_children`	Limit de memória, junto com `GOMEMLIMIT`
Exemplo	`11a`	`11b`	`11c`

7.1 Node.js

- **CPU:** o JavaScript roda em uma thread por processo, então um processo usa cerca de um core, mais algumas threads do garbage collector e do libuv. Defina o limit de CPU em 1 (2 se usar `worker_threads`) e escale com réplicas. Evite rodar uma frota de PM2 ou `cluster` dentro de um pod: cada worker tem seu próprio heap, e o consumo do pod em repouso cresce com o número de workers.
- **Tamanho do heap:** o V8 tira o limite do heap do limit de memória do container quando o processo inicia.
  - Lab: um limit de 512Mi deu um heap de 259 MiB no Node 24.22 e 268 MiB no Node 26.12. Um pod iniciado com 1Gi recebeu 560 MiB.
  - `NODE_OPTIONS=--max-old-space-size-percentage=75` subiu para 387 MiB com 512Mi. 70–75% deixa espaço para buffers, módulos nativos e o pool do libuv.
  - Depois de um resize in-place de 512Mi para 1Gi, o limite do heap continuou em 259 MiB (e 387 MiB com a flag), enquanto `process.constrainedMemory()` já mostrava o novo 1 GiB. Uma issue aberta no Node.js pede o recálculo dinâmico (nodejs/node#65235).
  - Consequência: encolha *requests* de memória in-place à vontade; mude *limits* de memória só com rollout.
- **Memória em repouso:** depois de um pico, o V8 manteve cerca de 120 MiB de lixo por uns 100 s, e então o RSS caiu de cerca de 200 MiB para 60 MiB sem nenhum tráfego. Um `global.gc()` forçado esvaziou o heap, mas não baixou o RSS na hora. Meça o RSS em repouso pelo menos 3 a 5 minutos depois que o tráfego para. O encolhedor do exemplo 05 usa o uso atual como piso de memória, então ele se adapta.
- **Tempo de inicialização:** defina `NODE_COMPILE_CACHE` (Node 22.1+) para o V8 reaproveitar código compilado. Um cache escrito em tempo de execução só ajuda em reinícios do container; gere o cache no build da imagem para que pods novos também se beneficiem (cache portável desde o Node 24.12, cache somente leitura desde o 26.8; docs). A inicialização é basicamente carregamento de módulos em uma thread, então um boost de CPU acima de 1 CPU ajuda pouco; um boost até 1 CPU ajuda quando o limit em regime é menor.
- **Probes:** um event loop com throttling responde tarde. Use timeouts generosos na liveness probe (o exemplo 11a usa 5 s e 6 falhas).
- **Versões:** o Node.js 24 é o LTS ativo (24.22.1), o 22 está em manutenção (22.23.4), o 26 é a versão current (26.12.1) e vira LTS no fim de outubro de 2026.

7.2 PHP

O PHP roda em um de dois modelos, e eles se comportam de forma bem diferente em repouso:

Modelo	Exemplos	Modelo de processos	Memória em repouso
Clássico, por requisição	nginx + PHP-FPM, FrankenPHP em modo clássico	A aplicação inicializa a cada requisição; workers são processos (FPM) ou threads	Pode cair a quase nada com `pm = ondemand`
Longa duração	FrankenPHP em worker mode, Laravel Octane (Swoole, RoadRunner, FrankenPHP)	A aplicação inicializa uma vez e fica na memória	Fica, como no Node.js

PHP-FPM:

- **Use `pm = ondemand`.** Os workers sobem quando chegam requisições e saem depois de `pm.process_idle_timeout` (padrão 10 s) sem trabalho (configuração do PHP-FPM). Lab, 8 clientes simultâneos por 20 s, cada requisição montando cerca de 24 MiB de strings pequenas:
  - `pm = static`, 8 workers: 172 MiB logo depois da carga, ainda 172 MiB em repouso.
  - `pm = ondemand`, até 8 workers: 172 MiB logo depois da carga, 0 workers e 10 MiB 25 s depois.
  - Com um único buffer grande por requisição, os dois modos voltaram ao normal depois de cada requisição, porque o PHP libera blocos grandes no fim da requisição. O ganho depende de como a aplicação aloca memória; meça a sua.
- **Custo do `ondemand`:** a primeira requisição depois de um período ocioso faz fork de um worker, o que leva milissegundos. O OPcache fica em memória compartilhada do master, então o código compilado sobrevive.
- **Limit de memória** ≥ `pm.max_children` × RSS de pico por worker + master e OPcache + folga. O `pm.max_children` é lido na inicialização; um resize in-place não o altera, então mude os dois juntos com rollout.
- **CPU:** uma requisição ocupa um processo worker. Defina o limit de CPU pela concorrência desejada por pod (1–2 CPUs) e escale com réplicas pela taxa de requisições.

PHP de longa duração:

- O FrankenPHP é um programa Go que embute o PHP. A documentação recomenda definir `GOMEMLIMIT` em containers, definir `num_threads`/`max_threads` explicitamente (o padrão é 2× os cores de CPU), manter `max_threads × memory_limit` abaixo da memória do container, e usar imagens com glibc em vez de Alpine/musl por desempenho (desempenho do FrankenPHP). As observações de Go da seção 7.3 também valem.
- O Octane e o worker mode do FrankenPHP trocam um caminho de requisição mais rápido por uma aplicação residente: dimensione e encolha esses serviços como serviços Node.js.

Filas: a fila Redis do Laravel guarda os jobs pendentes em uma lista Redis (`queues:default`), que o scaler `redis` do KEDA consegue observar para rodar workers a partir do zero (exemplo 14, seção 11).

Versões: PHP 8.5.12, 8.4.27 e 8.3.36 são as versões atuais (outubro de 2026).

7.3 Go

- **O GOMAXPROCS acompanha o limit de CPU** desde o Go 1.25 (release notes, blog):
  - O padrão é o limit de CPU arredondado para cima, mas nunca abaixo de 2 (a menos que a máquina tenha menos CPUs). Requests de CPU são ignorados.
  - O runtime reavalia no máximo uma vez por segundo. Lab: uma mudança in-place do limit 1 para 3 deu GOMAXPROCS=3 cerca de 1 s depois, sem reinício. De 2 para 1 ficou em 2, o piso documentado.
  - Isso só vale quando o `go.mod` declara `go 1.25` ou superior e a variável de ambiente `GOMAXPROCS` não está definida. Arquivos `go.mod` mais antigos mantêm o padrão antigo, o número de cores do nó. Em um nó de 64 cores isso significa 64 threads disputando uma cota de 1 CPU, o que causa throttling.
  - Consequência: defina um limit de CPU em todo container Go (ou defina `GOMAXPROCS`), e então encolha os requests à vontade.

- **O GOMEMLIMIT não é automático** do Go 1.25 ao 1.27. Um valor vindo da downward API é fixado quando o container inicia: depois de um resize in-place de 768Mi para 1Gi o runtime continuou reportando 768 MiB **(lab)**. Defina cerca de 90% do limit de memória explicitamente, ou use o pacote `automemlimit`.
- **Memória em repouso:** depois de um pico, o GC forçado do Go (a cada 2 minutos) esvaziou o heap, e o scavenger em segundo plano devolveu as páginas ao sistema cerca de 4,5 minutos depois do pico (157 → 10 MiB).
- **Inicialização:** binários estáticos em imagens `scratch` ou distroless são pequenos (5,9 MB no lab) e iniciam em milissegundos, então serviços Go são os melhores candidatos a scale-to-zero de verdade.
- **Versões:** Go 1.27.3 e 1.26.10 são as versões suportadas.

7.4 Cold start: o que os números do lab significam

Os tempos de 0,8–2,1 s para acordar do zero foram medidos através do interceptor HTTP do KEDA, com servidores mínimos, em um nó que já tinha a imagem. A maior parte desse tempo é do próprio Kubernetes: agendamento, criação do sandbox do pod, início do container e a primeira readiness probe (período de 1 s). Um cold start real soma:

Etapa	Tamanho típico	Como reduzir
Provisionamento de nó, se nenhum nó tem espaço	Minutos no OKE	Manter espaço livre com pods placeholder (seção 10.2); imagens OKE em vez de imagens de plataforma
Pull da imagem, se o nó não a tem	Segundos a dezenas de segundos	Imagens pequenas (distroless, Alpine), mesma imagem base entre serviços, pre-pull com um DaemonSet
Inicialização da aplicação	Boot do framework, conexões, aquecimento de cache	`NODE_COMPILE_CACHE`, preload do OPcache, inicialização lazy, startup boost
Readiness	Período da probe × falhas	`startupProbe` com período curto (1 s)

Meça cada aplicação antes de decidir, a partir dos timestamps do próprio pod:

```
kubectl -n <ns> get pod <pod> -o json \
  | jq -r '"created \(.metadata.creationTimestamp)", (.status.conditions[] | "\(.type) \
(.lastTransitionTime)")'
```

Se uma aplicação fica Ready em 1–2 s depois do `PodScheduled` e a imagem já está nos nós, escalá-la a partir do zero custa ao primeiro usuário cerca de 2–4 s. Se isso é aceitável é uma decisão de produto; a medição mostra a quais serviços isso se aplica.

8. Camada horizontal: nunca zero, raramente mais que o necessário

`examples/06-keda-cron-min-one.yaml`:

```
spec:
  minReplicaCount: 1              # um pod aquecido, sem cold start para o primeiro usuário
  maxReplicaCount: 10
  triggers:
    - type: cron                  # pré-aquecer antes de os usuários chegarem
      metadata: { timezone: America/Sao_Paulo, start: "15 7 * * 1-5", end: "0 20 * * 1-5",
desiredReplicas: "3" }
    - type: cpu
      metricType: AverageValue    # não Utilization: imune a mudanças de request
      metadata: { value: "400m" }
```

Lab: o KEDA criou um HPA com a métrica externa do cron e `cpu: AverageValue 400m`, como esperado.

Pontos práticos:

- **Pré-aquecer e restaurar.** Rode a restauração do horário comercial (5.4) e o scale-up do cron 15–30 minutos antes do tráfego chegar, para que a subida da manhã nunca espere um cold start.
- **Gatilhos por taxa de requisições** (scaler de Prometheus ou as métricas do KEDA HTTP add-on) acompanham a carga dos usuários melhor que a CPU.
- **Scale-out rápido a partir de 1** também precisa de capacidade de nó. Adicionar um nó no OKE leva minutos (imagens OKE reduzem o tempo de provisionamento em mais da metade em relação às imagens de plataforma). Para serviços sensíveis a latência, mantenha alguns pods "placeholder" de baixa prioridade: os pods reais os preemptam na hora, e o autoscaler adiciona um nó para os placeholders despejados em segundo plano (seção 10.2, exemplo `15`).
- **Não-produção pode ir a zero de verdade.** Ninguém espera um cold start de madrugada. Use o padrão documentado pela Oracle de cron do KEDA (+ HTTP add-on para acordar com a primeira requisição) e um node pool com mínimo 0 (blog da Oracle), ou o GoKubeDownscaler. A seção 11 lista quais workloads se encaixam e o que continua custando em zero.

9. Camada de nós: transformar requests menores em uma conta menor

9.1 Cluster Autoscaler (add-on do OKE)

Configuração: `examples/09-oke-cluster-autoscaler-addon.json`.

```
oci ce cluster install-addon --cluster-id <cluster-ocid> --addon-name ClusterAutoscaler \
  --from-json file://examples/09-oke-cluster-autoscaler-addon.json
```

Configurações que importam aqui (argumentos do add-on):

Chave	Padrão	Por que importa
`scaleDownUtilizationThreshold`	0,5	A "utilização" do nó é a soma dos **requests** / capacidade. É a redução dos requests que torna os nós removíveis. Com 0,6 os nós saem um pouco mais cedo
`scaleDownUnneededTime`	10m	Quanto tempo o nó precisa ficar abaixo do limiar
`skipNodesWithLocalStorage`	true	Pods com `emptyDir` bloqueiam o scale-down. Coloque false se o conteúdo do emptyDir for descartável
`nodes`		`min:max:pool-ocid`. Pools de aplicação em produção com mínimo ≥ 1, pools de não-produção com mínimo 0

Boa prática do OKE: mantenha um node pool fora do controle do autoscaler para os pods de sistema e para o próprio autoscaler (boas práticas). Problema conhecido: quando um pool sobe a partir de zero, o autoscaler superestima em 2x a CPU de shapes flex Arm, o que custa um ciclo extra de escalonamento (kubernetes/autoscaler#9337).

9.2 Karpenter Provider for OCI

`examples/08-karpenter-oci-burstable-and-preemptible.yaml` (validado contra os CRDs do KPO v1.4.0) define:

- um pool **low-cpu** em nós burstable E5/E4 Flex, com consolidação `WhenEmptyOrUnderutilized`;
- um disruption budget que bloqueia a consolidação das 07:00 às 19:00, de segunda a sexta (horário de São Paulo). Os schedules dos budgets do Karpenter não suportam fuso horário, então ficam escritos em UTC;
- um pool **preemptible** (`spot` no Karpenter, que o KPO mapeia para preemptible da OCI), que exige o taint `oci.oraclecloud.com/oke-is-preemptible`.

Comparado ao Cluster Autoscaler, o Karpenter consegue trocar um nó meio vazio por um menor e misturar shapes em uma única política. Isso importa quando os requests fora do expediente encolhem de forma desigual. Não deixe os dois autoscalers cuidarem dos mesmos pods (guia de migração).

9.3 Consolidação significa cold starts

Remover um nó significa fazer eviction dos pods dele, e cada pod despejado paga um cold start. Mitigações:

- Rode 2 réplicas com um PodDisruptionBudget (`maxUnavailable: 1`) no horário comercial. A segunda réplica sai barata depois do corte de CPU.
- Permita consolidação só na janela da noite (budgets do Karpenter), logo depois do encolhimento, com tráfego baixo.
- Mantenha o cold start curto com startup boost (5.3).
- Para um serviço de réplica única que nunca pode ser movido, anote com `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` ou `karpenter.sh/do-not-disrupt: "true"`. Esse nó passa a ficar, então use com moderação.

9.4 Alavancas de preço de nó para o pod aquecido (50m / 1Gi, 24x7)

Onde roda	USD/mês
E5 Flex, on-demand	2,12
E5 Flex burstable, baseline de 50%	1,84
E5 Flex burstable, baseline de 12,5%	1,64
A1 Flex (Arm), on-demand	1,54
Virtual node (Pod.Standard.E4.Flex)	20,44 + US$ 0,015/h por virtual node

- **Nós burstable** são overcommit de CPU com o preço já embutido pela OCI:
  - O Kubernetes vê todas as vCPUs, e você paga pelo baseline.
  - Se a CPU *média* do nó ficar abaixo do baseline, todo pod recebe o que pede. Se ficar acima, a OCI limita o nó inteiro e todos os pods ficam lentos.
  - Servem para não-produção e para serviços que passam a maior parte do tempo ociosos. O exemplo 08 também mostra a variante mais segura com `kubeReserved`, que esconde do scheduler a CPU acima do baseline.
  - Para um pod com CPU já reduzida o ganho é pequeno (a memória domina). O ganho é grande em pools onde os requests continuam altos e o uso é baixo.
- **Virtual nodes** não servem para pods pequenos. Requests precisam ser iguais aos limits, tamanhos abaixo de 1 core não são suportados, a alocação arredonda para OCPUs inteiras e GB inteiros mais 0,25 GB de overhead, e o VPA não funciona lá (alocação de recursos, comparação). São úteis como capacidade de pico stateless sobre os managed nodes (blog da Oracle sobre cluster misto).
- **Preemptible (−50%)** serve para a 2ª até a N-ésima réplica de serviços stateless, batch e CI. Nunca para a única réplica aquecida.
- **Arm (A1/A2)** tem a CPU mais barata, se suas imagens forem multi-arch.

9.5 Densidade de pods: quando endereços IP, e não CPU ou memória, limitam o empacotamento

Quando os pods ficam pequenos, o número de pods que um nó pode rodar pode virar o limite antes da CPU ou da memória:

- **Rede de pods VCN-native:** cada pod recebe um IP da VCN vindo das VNICs secundárias do nó. Máximo de pods por nó = MIN((VNICs − 1) × 31, 256) na configuração padrão (docs do OKE). VMs E5 Flex têm 2 VNICs com 1 OCPU e 1 VNIC por OCPU a partir de 2 OCPUs (tabela de shapes), então nós de 1 e 2 OCPUs

comportam 31 pods e um nó de 4 OCPUs, 93. Desde 2026 é possível anexar vários perfis de VNIC secundária e chegar a 256 pods por nó (release note).
- **flannel:** os pods usam uma rede overlay, e o número de pods não depende do shape (docs do OKE); vale o padrão de 110 do kubelet.

Custo por pod pequeno (requests de 25m de CPU / 128Mi, 4 vagas por nó ocupadas por DaemonSets, E5 Flex, 24x7):

Nó	USD/ mês	Pods que cabem por CPU/ memória	Máx. VCN-native	Máx. flannel	USD por pod, VCN-native	USD por pod, flannel
1 OCPU / 8 GB	33,58	53	31	110	1,24	0,69
2 OCPU / 16 GB	67,16	107	31	110	2,49	0,65
4 OCPU / 32 GB	134,32	214	93	110	1,51	1,27
8 OCPU / 64 GB	268,64	429	217	110	1,26	2,53

Com rede VCN-native, evite nós de 2 OCPUs para pools de pods pequenos. Com flannel, evite nós tão grandes que 110 pods deixem CPU e memória ociosas (ou aumente o `max-pods`). O CNI é escolhido na criação do cluster e não pode ser trocado depois (seção 12.4).

10. Tráfego imprevisível

Encolher por agenda (seção 5.4) é uma aposta de que o tráfego segue o horário comercial. O time espera isso, mas ainda não está confirmado. A configuração abaixo deixa barato perder essa aposta, e a seção 10.3 mostra como verificá-la com dados.

10.1 Deixar um palpite errado barato

- **Encolha requests, nunca limits.** O limit é o que o container pode usar; o request só importa quando o nó está cheio. **(lab)** Um pod com o request de CPU encolhido in-place para 25m, e limit de 1 CPU, ainda usou 0,91–0,99 CPU em um nó com CPU livre. Um pico noturno em um pod encolhido é atendido em velocidade total enquanto o nó tiver CPU sobrando.
- **O risco real é a disputa.** À noite os pods ficam mais apertados, e essa é justamente a economia. Se vários pods encolhidos tiverem pico ao mesmo tempo em um nó cheio, cada um recebe CPU proporcional ao request, que agora é pequeno. Três coisas cobrem isso:
  - Escalonamento horizontal com sinais absolutos (`AverageValue` de CPU, taxa de requisições, tamanho de fila; exemplos `06`, `07`). Réplicas novas nascem com o tamanho cheio do template, porque o encolhedor só altera pods em execução.
  - Espaço livre nos nós (10.2), para que essas réplicas novas subam em segundos em vez de esperar minutos por um nó.
  - Um startup boost (5.3), para que as réplicas novas passem rápido pela inicialização.
- **A memória continua segura.** O encolhedor nunca define um request de memória abaixo do uso atual + 15%, e os limits não mudam, então um pico não acrescenta risco de OOM kill.
- **Na dúvida, encolha menos.** Use o p95 medido fora do expediente, em vez do mínimo, como request ocioso. A economia cai um pouco e o risco de disputa cai bastante.

10.2 Capacidade de reserva com pods placeholder

`examples/15-capacity-placeholder.yaml` roda containers `pause` com prioridade negativa (-5) que reservam espaço nos nós. Quando um pod real não cabe, o scheduler despeja um placeholder e sobe o pod real na hora. O placeholder despejado fica Pending, e o autoscaler adiciona um nó para ele em segundo plano.

- **(lab)** Com um placeholder segurando 8,25 CPUs e só 0,5 CPU livre, um pod pedindo 2 CPUs ficou Ready 1 s depois de criado, preemptando o placeholder.
- A prioridade (-5) precisa ficar acima do `expendablePodsPriorityCutoff` do add-on Cluster Autoscaler (padrão -10). Caso contrário, um placeholder Pending não dispara scale-up.
- Você paga o espaço reservado o tempo todo. Dimensione-o como uma réplica de tamanho cheio do maior serviço por node pool. Se os picos noturnos forem raros, leve os placeholders a 0 à noite com as mesmas ferramentas da não-produção (seção 11).

10.3 Verificar a hipótese do horário comercial

Com o Prometheus coletando métricas do cAdvisor (por exemplo o kube-prometheus-stack, ou o Prometheus que o OpenCost exige), esta consulta retorna a fração da CPU usada fora do horário comercial nos últimos 7 dias. O Prometheus trabalha em UTC; 08:00–20:00 em São Paulo é 11:00–23:00 UTC. Validada no Prometheus 3.15:

```
sum by (namespace) (sum_over_time((
    sum by (namespace) (rate(container_cpu_usage_seconds_total{container!=""}[5m]))
    and on() (hour() < 11 or hour() >= 23 or day_of_week() == 0 or day_of_week() == 6)
  )[7d:5m]))
/
sum by (namespace) (sum_over_time(
    sum by (namespace) (rate(container_cpu_usage_seconds_total{container!=""}[5m]))
  [7d:5m]))
```

Fora do expediente são 108 das 168 horas da semana (64%). Se o uso de CPU fosse constante, a consulta retornaria cerca de 0,64. Quanto menor o valor, mais o tráfego se concentra no horário comercial. Faça o gráfico da expressão interna (sem o `sum_over_time`) no Grafana para ver o formato do dia e o tamanho dos picos noturnos. Se houver métricas do ingress, a mesma consulta sobre contagem de requisições é um sinal ainda melhor.

10.4 O que fazer com o resultado

O que os dados mostram	Vertical	Horizontal
Padrão claro de horário comercial (fração fora do expediente abaixo de cerca de 0,25; um ponto de partida, ajuste aos seus dados)	Encolher por agenda (`05`)	Pré-aquecimento por cron do KEDA + CPU absoluta (`06`)
Quase sempre calmo, com picos raros e imprevisíveis	Encolher por agenda + placeholders (`15`)	KEDA ou HPA por taxa de requisições; startup boost
Constante ou aleatório (fração perto de 0,64)	Só rightsizing com VPA `InPlaceOrRecreate` (`03`); sem agenda	HPA/KEDA por taxa de requisições ou `AverageValue`, `minReplicas` 1–2
Orientado a eventos (filas, jobs)	Não é necessário	Scaler de fila do KEDA com `minReplicaCount: 0` (`14`)

Serviços diferentes podem cair em linhas diferentes. Rode a consulta por namespace ou por workload.

11. O que pode ir a zero

11.1 O que "zero" significa

Scale-to-zero coloca o número de réplicas de um workload em 0, então nenhum pod roda. Isso sozinho não economiza nada no OKE, porque você paga por nós (seção 3). A economia vem no segundo passo: quando um node pool só rodava esses pods, o Cluster Autoscaler (pool com mínimo 0) ou o Karpenter remove os nós vazios, e o custo de computação deles para. Primeiro zero pods, depois zero nós.

O time descartou o zero para serviços de produção com usuários por causa do cold start. Vários outros workloads não têm ninguém esperando, e esses podem ir a zero hoje.

11.2 Candidatos

Workload	Por que zero é aceitável	Como	Exemplo
Ambientes de não-produção (dev, test, QA, staging, preview) fora do expediente	Ninguém usa à noite nem no fim de semana	GoKubeDownscaler: uma anotação por namespace	`13`
Aplicações de não-produção que alguém pode abrir à noite	Um usuário espera um cold start	Cron do KEDA + HTTP add-on (acorda com a requisição)	`12`
Ferramentas internas e painéis administrativos pouco usados, em qualquer ambiente	Um usuário pode esperar alguns segundos uma vez	KEDA HTTP add-on	`12`
Workers e consumidores de fila, em todos os ambientes, inclusive produção	O "usuário" deles é uma fila; alguns segundos de atraso não fazem diferença	Scaler de fila do KEDA, `minReplicaCount: 0`	`14`
Jobs agendados	Rodam e terminam	CronJob do Kubernetes (já fica em zero entre execuções)	
Serviços de produção com usuários	Não aceitável por enquanto (requisito do time)	Manter 1 pod aquecido e mínimo (seções 5–10)	`05`, `06`

Resultados do lab:

- **Downscaler de namespace (GoKubeDownscaler 1.4.2):** com `downscaler/uptime: "Mon-Fri 07:30-20:00 America/Sao_Paulo"` no namespace, um Deployment de 3 réplicas foi a 0 fora da janela e voltou a 3 cerca de 30 s depois do início do uptime; um workload marcado com `downscaler/exclude: "true"` continuou rodando. Dois padrões que vale conhecer: workloads novos são ignorados por um período de carência de 15 minutos, e o Helm chart só trata Deployments, a menos que você inclua `statefulsets` em `includedResources`.
- **Worker de fila (scaler `redis` do KEDA):** o primeiro worker ficou Ready 2–10 s depois da chegada do primeiro job (o KEDA consulta a cada 10 s no exemplo), o worker escalou para 5 réplicas com 100 jobs, e voltou a 0 cerca de 2–3 minutos depois que a fila esvaziou (`cooldownPeriod: 120`).
- **Acordar com a requisição (KEDA HTTP add-on 0.16, beta):** a primeira requisição a uma aplicação com zero réplicas levou 1,0–1,7 s para o nginx, e 0,8–2,1 s para servidores mínimos de Node.js, PHP-FPM e Go, em um nó que já tinha a imagem.

Observações:

- O KEDA 2.21 não tem scaler específico da OCI. Use o scaler do broker que vocês rodarem (Redis, RabbitMQ, Kafka, PostgreSQL, MySQL e outros), ou os scalers `prometheus` / `metrics-api`. O OCI Streaming expõe um endpoint compatível com Kafka que pode funcionar com o scaler Kafka do KEDA; não testei.
- O KEDA, o interceptor do HTTP add-on e o autoscaler precisam rodar em um node pool que nunca vai a zero, como no guia da Oracle de KEDA + Cluster Autoscaler.
- Com o HTTP add-on, o tráfego precisa passar pelo proxy do interceptor. Aponte o Ingress ou Gateway dessas aplicações para ele.

11.3 O que continua custando dinheiro em zero

Item	USD/mês
Control plane de cluster enhanced (cluster basic não tem taxa)	73,00
Node pool de sistema, 2 × E5 Flex 1 OCPU / 8 GB com boot volumes de 50 GB	71,41
Node pool de sistema com um único nó (só não-produção, sem redundância)	35,70
Flexible load balancer, cada um além das horas gratuitas	8,25
Network load balancer	0,00
Volume persistente, 50 GB, desempenho balanced, cada	2,12
Node pool de aplicação com 0 nós	0,00

- A tabela de preços dá as primeiras 744 horas de load balancer de cada mês grátis, o que equivale a um load balancer rodando o mês inteiro. O OCI Network Load Balancer é gratuito (Oracle); o OKE cria um para um `Service` do tipo `LoadBalancer` com a anotação `oci.oraclecloud.com/load-balancer-type: "nlb"` (docs do OKE). Ele trabalha só na camada 4, então TLS e roteamento HTTP ficam no ingress ou gateway dentro do cluster.
- Cada cluster enhanced separado custa US$ 73/mês mesmo vazio. Rode os ambientes de não-produção como namespaces em um cluster compartilhado, ou como clusters basic (seção 12.1).
- Volumes persistentes e seus dados continuam sendo cobrados enquanto os pods estão em zero.

11.4 Obter os números por ambiente

- **Do lado da OCI:** dê a cada ambiente o seu próprio node pool e aplique uma defined tag de cost-tracking aos nós do node pool (as node tags são aplicadas às instâncias quando elas são criadas; nós existentes mantêm as tags antigas até serem substituídos) (node tags do OKE). O Cost Analysis da OCI consegue então agrupar o custo de computação por essa tag. Um compartimento por ambiente também funciona.
- **Do lado do Kubernetes:** o OpenCost suporta preços da OCI e divide o custo dos nós por namespace, workload e label, inclusive entre ambientes que compartilham nós (OpenCost na OCI).
- Com a divisão em mãos: se os ambientes de não-produção rodam a semana inteira hoje, levá-los a zero fora do expediente economiza perto de dois terços do custo de computação deles (a fração da semana fora do horário comercial). Os itens fixos da seção 11.3 continuam.

12. Escolha da variante de cluster do OKE

O time ainda não definiu a configuração do cluster. Estas são as escolhas que afetam esta estratégia, e quais delas não podem ser mudadas depois.

12.1 Cluster basic ou enhanced

	Basic	Enhanced
Preço do control plane	Grátis	US$ 0,10/h, no máximo US$ 74,40/mês
Compromisso de disponibilidade	Objetivo de nível de serviço (SLO)	SLA com garantia financeira para o API server
Add-ons gerenciados pelo OKE (Cluster Autoscaler, Metrics Server, versões do CoreDNS)	Não: vocês instalam e atualizam	Sim
Workload identity (IAM por service account do Kubernetes)	Não: instance principals, que concedem mais permissões que o necessário	Sim
Virtual nodes, self-managed nodes	Não	Sim

	Basic	Enhanced
Node cycling em atualizações de node pool e upgrades do Kubernetes	Não	Sim
Nós de trabalho por cluster	Limite menor	5.000; até 20.000 com aprovação da Oracle
Karpenter Provider for OCI	Não com a configuração documentada, que usa workload identity	Sim
Mudar depois	Pode virar enhanced se o endpoint da API for VCN-native; só nessa direção	Não pode voltar a basic

Fontes: comparação, clusters enhanced, upgrade.

O que isso significa aqui:

- A maior parte deste relatório funciona também em cluster basic: in-place resize, VPA, KEDA, GoKubeDownscaler, o encolhedor, e o Cluster Autoscaler como deployment standalone com instance principals. O que se perde são os add-ons gerenciados, o workload identity, o Karpenter Provider for OCI e os virtual nodes.
- Para produção, os US$ 73/mês são pouco perto do custo dos nós, e compram o SLA, o workload identity e o node cycling.
- Para não-produção, prefira um cluster enhanced compartilhado com um namespace por ambiente em vez de vários clusters. Se precisarem mesmo de muitos clusters de vida curta, clusters basic evitam US$ 73/mês cada. Crie-os com endpoint de API VCN-native para poder fazer upgrade depois.

12.2 Tipos de nó

	Managed nodes	Virtual nodes	Self-managed nodes
O que se paga	Instâncias de computação (OCPU + GB)	Cada pod, arredondado para OCPUs e GB inteiros, mais US$ 0,015/h por virtual node	Instâncias de computação
Requests abaixo dos limits	Sim	Não: requests precisam ser iguais aos limits	Sim
In-place resize e VPA	Sim	VPA não suportado; cada pod tem alocação fixa	Sim
DaemonSets	Sim	Não	Sim
Burstable / preemptible	Preemptible em node pools; burstable via Karpenter	Não se aplica	Os dois (vocês criam as instâncias)
Autoscaling de nós	Cluster Autoscaler ou Karpenter	Não é necessário	Karpenter ou ferramenta própria
Tipo de cluster	Basic ou enhanced	Enhanced	Enhanced
Adequação a esta estratégia	Escolha padrão	Capacidade de pico para pods stateless de 1 OCPU ou mais	Shapes ou imagens especiais; mais trabalho de operação

Fonte: virtual vs managed nodes. O custo de virtual nodes para pods pequenos está na seção 9.4.

12.3 Autoscaler de nós

	Cluster Autoscaler, add-on do OKE	Cluster Autoscaler, standalone	Karpenter Provider for OCI
Tipo de cluster	Enhanced	Basic ou enhanced	Enhanced
Shapes	Um por node pool	Um por node pool	Vários, escolhidos por pod a partir de uma política

	Cluster Autoscaler, add-on do OKE	Cluster Autoscaler, standalone	Karpenter Provider for OCI
Pools com zero nós	Sim (mínimo 0 no pool)	Sim	Sim (sem pools fixos)
Remove nós subutilizados	Quando os requests ficam abaixo de `scaleDownUtilizationThreshold`	Igual	Consolidação: remove ou troca por nós menores
Burstable / preemptible	Node pools preemptible	Igual	Os dois
Situação	Gerenciado pela Oracle	Projeto upstream	GA desde abril de 2026 (v1.4.0)
Quando usar	Comece por aqui	Em clusters basic	Quando precisar de vários shapes, nós burstable ou janelas de consolidação

Não deixe dois autoscalers cuidarem dos mesmos pods (seção 9.2).

12.4 Rede de pods (CNI)

	flannel	Rede de pods VCN-native
Endereços IP dos pods	Rede overlay, visível só dentro do cluster	IPs reais da VCN, alcançáveis de VCNs pareadas e on-premises
Pods por nó	Não depende do shape (padrão do kubelet: 110)	MIN((VNICs − 1) × 31, 256); 31 em nós E5 de 1–2 OCPUs
IPs da VCN usados	Nenhum para pods	Um por pod, pré-alocados por nó
Regras de rede por subnet de pods (NSGs), VCN flow logs, Zero Trust Packet Routing	Não	Sim
Virtual nodes	Não suportado	Obrigatório
Padrão ao criar um cluster	API	Console
Mudar depois	Impossível: o CNI é fixado na criação do cluster (docs)	Impossível

Para esta estratégia, a densidade de pods importa (seção 9.5): com pods pequenos, o flannel empacota até 110 por nó em qualquer shape, enquanto o VCN-native exige nós maiores ou perfis extras de VNIC. Escolha VCN-native se precisarem de pods endereçáveis a partir da VCN ou on-premises, isolamento de pods por NSG, ou virtual nodes.

12.5 Versão do Kubernetes

Versão no OKE	In-place resize	Observações
1.34.10	Beta, ligado por padrão	A mais antiga suportada; sai de suporte uns 30 dias depois que a próxima versão chegar
1.35.2	GA	Exige cgroup v2 (imagens OKE OL8)
1.36.4	GA; resize de recursos no nível do pod em beta	A versão GA mais nova no OKE; Karpenter Provider for OCI v1.3.0+
1.37.0	GA	Só em preview no OKE; HPA scale-to-zero vira beta

O OKE suporta três versões por vez; a 1.33 perdeu suporte em 10 de agosto de 2026. Comece clusters novos na 1.36 e planeje um upgrade a cada poucos meses.

12.6 Se vocês puderem escolher

- **Produção:** cluster enhanced; Kubernetes 1.36; managed nodes em E5 ou E6 Flex (A1/A2 Arm se as imagens forem multi-arch); flannel, a menos que precisem de VCN-native, e nesse caso nada de nós de 2 OCPUs para

pods pequenos; um node pool de sistema pequeno fora do controle do autoscaler; primeiro o add-on Cluster Autoscaler, depois o Karpenter para nós burstable e janelas de consolidação.

- **Não-produção:** um cluster compartilhado com um namespace por ambiente; um node pool de aplicação com mínimo 0 em capacidade preemptible; GoKubeDownscaler nos namespaces; KEDA para workers e aplicações que acordam com a requisição.

13. Plano de adoção sugerido

Fase	Ações	Critério de saída
0. Medir (1–2 semanas)	Todo container com requests e limits. Instalar Metrics Server (add-on), VPA em modo `Off` (`02`), Prometheus, e OpenCost ou Goldilocks. Aplicar tags aos node pools por ambiente (11.4). Medir o cold start de cada aplicação (7.4). Rodar a consulta do horário comercial (10.3)	CPU/memória em repouso e no pico conhecidas por serviço; custo por ambiente visível; fração fora do expediente conhecida
1. Rightsizing	Aplicar as recomendações do VPA, via GitOps ou `InPlaceOrRecreate` + `RequestsOnly` (`03`). Aplicar as configurações de runtime da seção 7 (`11a`, `11b`, `11c`). Mudar os HPAs para `AverageValue` ou taxa de requisições	Requests a ~20% do p90 de uso; sem OOM kills nem alertas de throttling
2. Zero onde ninguém espera	Namespaces de não-produção a zero fora do expediente (`13`); workers de fila a zero (`14`); ferramentas internas acordando com a requisição (`12`)	Node pools de não-produção com 0 nós à noite; sem risco para produção
3. Aquecido e mínimo em produção	KEDA `minReplicaCount: 1` + pré-aquecimento por cron (`06`); encolhedor ocioso (`05`) ou VPA, não os dois; startup boost (`04` ou `10`); placeholders se os picos forem imprevisíveis (`15`). Escolher a linha da tabela 10.4 por serviço	Requests de CPU à noite ≥ 80% menores; latência da primeira requisição inalterada
4. Capacidade	Ajustar o add-on do Cluster Autoscaler (`09`) ou adotar o Karpenter com janelas de consolidação (`08`); pool burstable para serviços de pouca CPU; preemptible para réplicas extras; tamanhos de nó adequados ao limite de densidade de pods (9.5)	O número de nós acompanha os requests em até ~30 min após o encolhimento
5. Cold start	Tamanho da imagem, compile cache e preload (7.4)	Se o cold start de um serviço ficar curto o bastante, considerar o zero de verdade para ele

A fase 2 pode começar em paralelo com a fase 1: ela não exige ajuste dos serviços de produção.

14. Riscos e ressalvas

Risco	Mitigação
Request de memória abaixo do uso real → eviction precoce sob pressão de memória	Request ≥ uso em repouso + 10–15% (o encolhedor garante isso); PriorityClass para serviços críticos
Limit de memória reduzido abaixo do uso → OOM kill	Nunca reduzir limits in-place; VPA com `RequestsOnly`
Heap do Node.js dimensionado pelo limit de memória da inicialização	Mudar limits de memória só com rollout; definir `--max-old-space-size-percentage`
`pm.max_children` × tamanho do worker do PHP-FPM acima do limit de memória → OOM kill sob carga	Dimensionar o limit pelo RSS medido dos workers; mudar os dois juntos
Serviço Go com `go 1.24` ou anterior no `go.mod`, ou sem limit de CPU → GOMAXPROCS = cores do nó, throttling	`go 1.25`+ no `go.mod` e limit de CPU em todo container Go, ou definir `GOMAXPROCS`
Avalanche de scale-out do HPA depois de encolher requests	Só métricas `AverageValue` ou de taxa de requisições
Pico noturno inesperado em um nó apertado	Encolher só requests; escalar horizontalmente com métricas absolutas; placeholders (10.2)
Consolidação despeja o pod aquecido → cold start	2 réplicas + PDB durante o dia; janelas de consolidação; startup boost

Risco	Mitigação
Pods pequenos batem no limite de pods por nó (VCN-native)	Tamanhos de nó da seção 9.5, perfis extras de VNIC, ou flannel em clusters novos
Nó burstable acima do baseline por muito tempo → todos os pods lentos	Só pools de pouca CPU; acompanhar a métrica `CpuUtilization` da OCI; ou usar a variante com `kubeReserved`
Recursos alpha e beta (VPA `InPlace`, `CPUStartupBoost`, KEDA HTTP add-on)	Testar em não-produção; em produção preferir `InPlaceOrRecreate`, kube-startup-cpu-boost e os scalers do núcleo do KEDA
O downscaler ignora workloads novos (carência de 15 minutos) e StatefulSets por padrão	Conhecer os padrões; incluir `statefulsets` em `includedResources` no chart
Pools de threads da JVM dimensionados para um limit de CPU aumentado	Boost só nos requests (`RequestsOnly`), fatores modestos ou `-XX:ActiveProcessorCount`
O lab usou containerd, o OKE usa CRI-O	Repetir os testes in-place em um cluster OKE de desenvolvimento antes de produção

15. Perguntas em aberto para o time

Respondidas em 27 de outubro: runtimes (seção 7), formato do tráfego (seção 10), o que pode ir a zero (seção 11) e variantes de cluster (seção 12). Ainda em aberto:

1. **Cold start por aplicação.** Quais frameworks (por exemplo Express, NestJS ou Next.js; Laravel ou Symfony, em PHP-FPM ou Octane/FrankenPHP)? Quanto cada um leva de `PodScheduled` a `Ready` (7.4)? Isso define onde o zero de verdade passa a ser aceitável.
2. **Dados de tráfego.** Depois de 1–2 semanas de dados no Prometheus, o que a consulta da seção 10.3 retorna por namespace?
3. **Custo por ambiente.** Depois de aplicar tags aos node pools (11.4), quanto do gasto é não-produção?
4. **Requisitos de rede.** Os pods precisam de IPs roteáveis na VCN, isolamento por NSG, Zero Trust Packet Routing ou virtual nodes? Isso define o CNI, que não pode mudar depois que o cluster é criado.
5. **Filas.** Haverá workers em segundo plano, e em qual broker (Redis, RabbitMQ, Kafka ou OCI Streaming, OCI Queue)? Isso define o scaler do KEDA.
6. **Preço.** Pay-as-you-go ou tarifa negociada? As proporções continuam as mesmas, a economia absoluta muda.

Apêndice A. Arquivos

Caminho	Conteúdo
`examples/01-app-deployment.yaml`	Workload com QoS Burstable pronto para in-place resize, PDB, labels de opt-in
`examples/02-vpa-recommend-only.yaml`	VPA em modo `Off`
`examples/03-vpa-inplace.yaml`	VPA `InPlaceOrRecreate`, `RequestsOnly`, mín/máx
`examples/04-vpa-startup-boost.yaml`	Igual ao 03 mais CPU Startup Boost (alpha)
`examples/05-idle-shrinker-cronjob.yaml`	RBAC + script + CronJobs para encolher in-place fora do expediente. Revisão 2: os containers são alterados na ordem do pod (a API de resize rejeita reordenação), e um pod com falha não interrompe mais a execução
`examples/06-keda-cron-min-one.yaml`	ScaledObject do KEDA: mínimo 1, pré-aquecimento por cron, CPU absoluta

Caminho	Conteúdo
`examples/07-hpa-absolute-cpu.yaml`	HPA simples com `AverageValue`
`examples/08-karpenter-oci-burstable-and-preemptible.yaml`	OCINodeClass/NodePool do KPO: pool burstable com janelas de consolidação, pool preemptible
`examples/09-oke-cluster-autoscaler-addon.json`	Configuração do add-on Cluster Autoscaler do OKE
`examples/10-kube-startup-cpu-boost.yaml`	Recurso StartupCPUBoost
`examples/11a-runtime-nodejs.yaml`	Deployment Node.js: percentual de heap, compile cache, probes, anotações do encolhedor
`examples/11b-runtime-php-fpm.yaml`	Deployment PHP-FPM + nginx com `pm = ondemand`
`examples/11c-runtime-go.yaml`	Deployment Go: limit de CPU para o GOMAXPROCS, `GOMEMLIMIT` explícito
`examples/12-nonprod-wake-on-request.yaml`	Cron do KEDA + HTTP add-on: zero à noite, acorda com a primeira requisição
`examples/13-nonprod-namespace-downscaler.yaml`	Anotação de namespace do GoKubeDownscaler, com um workload excluído
`examples/14-keda-queue-worker-zero.yaml`	Worker de fila que escala 0 → N → 0 a partir de uma lista Redis
`examples/15-capacity-placeholder.yaml`	PriorityClass + pods placeholder para espaço livre nos nós
`lab/run-lab.sh`	Cria o cluster kind e reproduz os resultados **(lab)** da revisão 1; `--delete` remove o cluster
`lab/run-runtime-lab.sh`	Reproduz os resultados da revisão 2 (runtimes, zero, placeholders); `--long` acrescenta as medições de memória em repouso
`lab/runtime-*.yaml`, `lab/redis.yaml`	Pods de teste usados pelo `run-runtime-lab.sh`
`lab/cost-model.py`	Gera as tabelas de custo das seções 6, 9 e 11

Os comentários dentro dos manifestos estão em inglês.

Apêndice B. Fontes

- OKE: versões suportadas, release notes, Kubernetes 1.35 no OKE, VPA no OKE, Cluster Autoscaler, argumentos do add-on CA, boas práticas de autoscaler, KPO, GA do KPO, KEDA + CA no OKE, recursos de virtual nodes, virtual vs managed nodes, nós preemptible.
- Variantes de cluster e rede do OKE: basic vs enhanced, clusters enhanced, upgrade de basic para enhanced, clusters de 20.000 nós, rede de pods, CNI VCN-native, CNI flannel, várias VNICs secundárias, node tags, network load balancers.
- OCI Compute e preços: instâncias burstable, cobrança burstable, instâncias preemptible, definição de OCPU do Ampere A2, Flexible Network Load Balancer, API de preços.
- Kubernetes: in-place resize, blog do GA na 1.35, resize de recursos no nível do pod (beta 1.36), HPA scale-to-zero beta, node-pressure eviction.
- VPA: features, flags, limitações conhecidas, AEP-7862 CPU startup boost, release do VPA 1.8.0.
- Node.js: opções de CLI (`--max-old-space-size-percentage`), compile cache de módulos, memória do Node.js 20+ em containers (Red Hat), nodejs/node#65235, índice de versões.
- PHP: configuração do PHP-FPM, worker mode do FrankenPHP, desempenho do FrankenPHP, versões suportadas.

- Go: release notes do Go 1.25, GOMAXPROCS ciente de containers, pacote runtime (GOMEMLIMIT), downloads.
- Ferramentas: KEDA, KEDA HTTP add-on, GoKubeDownscaler, kube-startup-cpu-boost, Kedify PodResourceProfile, EPCAS, OpenCost na OCI, disruption no Karpenter.

Apêndice C. Glossário de unidades

As quantidades de CPU e memória neste relatório usam a notação do Kubernetes, a mesma dos manifestos de exemplo. Os preços usam as unidades de cobrança da OCI. Fontes: unidades de recursos do Kubernetes, tabela de preços da OCI.

C.1 CPU

Termo	Significado	Exemplo
CPU (Kubernetes)	Uma CPU em um request ou limit é uma CPU virtual (vCPU) do nó	`cpu: "1"` = uma vCPU inteira
m (milicore)	Um milésimo de CPU; `1000m` = 1 CPU. Também chamado de millicpu	`50m` = 0,05 CPU, 5% de uma vCPU. `250m` = um quarto. `500m` e `0.5` são o mesmo valor: meia CPU
vCPU	Uma thread de hardware. O Linux e o Kubernetes a contam como uma CPU	Um nó com 4 vCPUs comporta pods cujos requests somam um pouco menos de `4` (o sistema reserva uma parte)
OCPU	A unidade de cobrança de CPU da Oracle: um core físico	Shapes x86 (E4, E5, E6): 1 OCPU = 2 vCPUs. Ampere A1: 1 OCPU = 1 vCPU. AmpereOne A2: 1 OCPU = 2 cores de uma thread = 2 vCPUs
Request de CPU	CPU reservada para o container quando o scheduler o coloca em um nó. O modelo de custo precifica os requests	30 pods × `50m` reservam 1,5 vCPU
Limit de CPU	O máximo de CPU que o container pode usar, aplicado em períodos de 100 ms	`1` = até 100 ms de tempo de CPU a cada 100 ms. Acima disso o container sofre throttling (fica mais lento), não é morto
Baseline (nós burstable)	A fração de cada OCPU que uma instância burstable da OCI pode usar o tempo todo. É também a fração cobrada	2 OCPUs com baseline de 50% são cobradas como 1 OCPU

C.2 Memória

Como aparece	Nome	Tamanho
`Ki`, `Mi`, `Gi`	kibibyte, mebibyte, gibibyte (unidades binárias)	Potências de 1024. 1Mi = 1.048.576 bytes; 1Gi = 1024Mi = 1.073.741.824 bytes
`k`, `M`, `G`	kilobyte, megabyte, gigabyte (unidades decimais)	Potências de 1000. 1M = 1.000.000 bytes; 1G = 1.000.000.000 bytes
MiB, GiB	O mesmo que `Mi`, `Gi`	O relatório escreve assim os valores medidos, por exemplo "RSS de 157 MiB"
MB, GB	O mesmo que `M`, `G`	A OCI cobra memória por GB, e tamanhos de nó como "8 GB" são números da OCI

- `512Mi` é meio gibibyte (cerca de 537 MB). `1Gi` é cerca de 1,074 GB. `256M` é cerca de 5% menor que `256Mi`; os dois são válidos em um manifesto.
- **Maiúsculas e minúsculas importam.** `M` maiúsculo significa megabytes, mas `m` minúsculo significa mili: `400m` de memória são 0,4 *bytes*. O Kubernetes aceita sem erro. Como request, reserva quase nada; como limit, o container não consegue iniciar. Para memória, escreva `400Mi`.
- **Request de memória:** memória reservada no nó quando o pod é agendado. O container pode usar mais, mas aí fica entre os primeiros a sofrer eviction quando falta memória no nó (seção 6.3).
- **Limit de memória:** teto rígido. Acima dele o container sofre OOM kill (é parado pelo kernel por usar memória demais) e é reiniciado.

- **RSS (resident set size):** a memória que um processo realmente ocupa na RAM. As medições de memória em repouso das seções 6.4 e 7 são RSS, ou o uso total de memória do container (`memory.current`).
- **Heap:** a memória que o runtime de uma linguagem gerencia para os objetos do programa (o V8 no Node.js, o runtime do Go). O máximo é definido por `--max-old-space-size` no Node.js e limitado de forma flexível pelo `GOMEMLIMIT` no Go. Ele precisa ficar abaixo do limit de memória do container.

C.3 Preços e tempo

Termo	Significado
USD, US$	Dólares americanos, pela tabela pay-as-you-go da OCI em 27 de outubro de 2026, sem descontos nem impostos
OCPU-hora	Preço de uma OCPU por uma hora. E5 Flex: US$ 0,030
vCPU-hora	O preço da OCPU-hora dividido pelas vCPUs de uma OCPU. E5 Flex: US$ 0,030 / 2 = US$ 0,015. O relatório usa esse valor porque o Kubernetes conta vCPUs
GB-hora	Preço de 1 GB de memória por uma hora. E5 Flex: US$ 0,002
Mês	730 horas (365 dias × 24 horas / 12 meses). Todo valor em USD/mês neste relatório é um preço por hora × 730. Os tetos mensais e franquias gratuitas da Oracle usam 744 horas (31 dias); por exemplo, a taxa do cluster enhanced tem teto de US$ 74,40
24x7	Rodando todas as horas do mês: 730 horas
Horário comercial, fora do expediente	Horário comercial é de segunda a sexta, 08:00–20:00 no horário de São Paulo: 60 das 168 horas da semana (36%). Fora do expediente são as outras 108 horas (64%)
Custo de um pod	O custo da capacidade de nó que os *requests* dele reservam, não do que ele realmente usa. Isso supõe que o autoscaler de nós mantém os nós bem empacotados (seção 2)

C.4 Exemplo calculado: quanto custa um pod aquecido

Um pod com requests de `50m` de CPU e `1Gi` de memória em nós E5 Flex, rodando 24x7:

Parte	Cálculo	USD/mês
CPU	`50m` = 0,05 vCPU. 0,05 × US$ 0,015 por vCPU-hora × 730 horas	0,55
Memória	`1Gi` = 1,074 GB. 1,074 × US$ 0,002 por GB-hora × 730 horas	1,57
Total		2,12

- Com o request de CPU em `500m`, a parte de CPU fica 10 vezes maior: 0,5 × 0,015 × 730 = US$ 5,47, e o pod custa US$ 7,04 por mês. A CPU passa a ser 78% do custo (5,47 / 7,04), o número usado nas seções 1 e 6.2.
- O número "7,5 GB de RAM custam o mesmo que 1 vCPU" da seção 6.1 é US$ 0,015 / US$ 0,002.
- Os cenários de 30 serviços da seção 6.5 aplicam a mesma conta para cada hora da semana, e depois multiplicam pelo número de pods e de serviços.
- **Premissa sobre a unidade de memória:** a tabela de preços da OCI diz "gigabyte", então o modelo de custo converte com 1Gi = 1,074 GB. Se a OCI na verdade contar memória em gigabytes binários (algumas tabelas da Oracle, como a do Compute Cloud@Customer, dizem "gibibyte"), os valores de memória ficam cerca de 7% menores. As conclusões não mudam: a CPU seria uma parte ainda maior do custo.

C.5 Outros termos usados com números

Termo	Significado
p90, p95	Percentis 90 e 95: 90% (ou 95%) dos valores medidos ficam neste valor ou abaixo dele. O VPA define os requests de CPU a partir do p90 do uso
Réplica	Uma cópia de um pod. "3 réplicas" significa três pods idênticos dividindo o tráfego
Fração fora do expediente	A parte do uso de CPU da semana que acontece fora do horário comercial (seção 10.3). 0,64 significa uso constante; valores menores significam uso concentrado no horário comercial

Termo	Significado
`Utilization` e `AverageValue`	Duas formas de definir o alvo de CPU de um HPA: como percentual do request (`Utilization`), ou como quantidade absoluta, por exemplo `400m` (`AverageValue`). A seção 5.5 explica por que este relatório usa a forma absoluta